

Particle Filter Localisation -- Change Log

Testing Infrastructure and Visualisation Tools

Built a standalone test and analysis framework that exercises the particle filter's core components outside the ROS 2 build system. This allows rapid iteration on geometry, resampling, and seeding logic without needing the full Booster robot stack.

- **Header-only template extraction for testability.** The resampling pipeline (`MergeSeededParticles.hpp`, `ResampleParticles.hpp`) and seeding geometry (`SeedGeometry.hpp`) were extracted into header-only templates that can be compiled independently. These are included by both the production `ParticleFilter.cpp` and the standalone test harness, guaranteeing zero code duplication.
- **Particle filter unit tests (17 scenarios).** A standalone C++ test program (`test-runs/particle-filter-unit-tests/main.cpp`) exercises 10 merge-seeded scenarios (M1-M10) and 7 resampling scenarios (R1-R7), each testing edge cases like empty candidate lists, single candidates, weight extremes, and degenerate particle counts. Tests pass/fail are printed to stdout, and CSV output can be piped into Python visualisation scripts.
- **Seed geometry correctness tests.** A geometry-focused test harness (`test-runs/seed-geometry-test/main.cpp`) creates synthetic observation scenarios from known robot poses and field features, then runs `computeSeedPose()` in reverse to verify that the implied robot pose matches the ground truth to numerical precision. Results are saved to CSV for visual inspection and plotted as overview diagrams.
- **CSV logging from the running particle filter.** The filter's debug logging system writes per-tick particle snapshots (`pf_all_particles.csv`), tick metadata (`pf_tick_metadata.csv`), and seed event details (`pf_seed_events.csv`). These logs capture the pre-observation, post-seeding, and post-resampling states for every tick, enabling offline analysis.
- **Offline playback visualiser.** A Python visualiser (`test-runs/particle-filter-playback/test-1/visualize_pf_playback.py`) loads the CSV logs and renders particles on a RoboCup field layout, colour-coded by particle type (standard, seeded, fill), with the estimated pose overlaid. Supports interactive frame-by-frame stepping, play/pause animation, and seed event overlays. Can also render video output for presentation use.
- **Seeded particle pose analysis.** A set of Python analysis scripts (`test-runs/seeded-particle-pose-visualisation/`) process seed candidate data across multiple visualisation runs, computing RMSE between candidate poses and ground truth, checking candidate plausibility against field geometry, and producing histograms and scatter plots. The analysis confirmed that seed candidates are geometrically correct inversions of observations, but that noisy observations themselves were the root cause of incorrect seeding -- directly motivating the observation-scaled noise and field-boundary plausibility checks described below.

Changes on `PAR/particle_filter_based_localisation` since 28 May 2026

Isolate Particle Resampling and Improve Testability (PR #305)

- Extracted `mergeSeededParticles()` and `resampleParticles()` into header-only template files so the core resampling pipeline can be unit tested without linking the full robot codebase. This makes regression testing faster and more reliable.
- Created a standalone `ParticleLite` type with float fields to decouple test code from the full `Particle` class. Test harnesses no longer need to construct production-grade particles.
- Isolated the seeding geometry logic into `SeedGeometry.hpp`, enabling targeted unit tests for the geometric calculations used when generating seed candidates from field feature observations.
- Added a minimum particle threshold to prevent the filter from degenerating below a viable population size when too many particles are culled. This avoids scenarios where the particle set collapses and cannot recover.
- Fixed a mismatch between directional and directionless field feature handling for goal posts. The filter was previously treating goal posts as directionless features when they carry directional information, causing incorrect likelihood assignments.
- Changed the resampling behaviour so existing particles are resampled (drawn with replacement proportional to weight) rather than always generating entirely new particle sets. This preserves diversity from the previous generation and improves convergence stability.

Uncertainty Corrections and Parameter Tuning (PR #326)

- Fixed the uncertainty computation in the particle filter's pose output. The earlier calculation was not correctly propagating observation noise through to the final estimate, which made the reported confidence unreliable for downstream systems like the game controller and behaviour engine.
- Added logging decimation configuration to reduce the verbosity of particle filter diagnostic output. This prevents log files from growing too large during extended runs while still capturing enough data for offline analysis.
- Adjusted several motion model parameters to better match the Booster K1's actual odometry characteristics, reducing drift between the internal model and real-world robot displacements.

Seeding Feedback Loop Remediation (Branches 1, 3, 4, 5 of the remediation plan)

- Broke the positive feedback loop in the seeding system by introducing temporal gating. The filter now waits a minimum number of ticks between successive seed events, preventing vision-based seed proposals from reinforcing the same incorrect hypothesis frame after frame.
- Added field-boundary plausibility checks in `seedFromDirectionalObservation()`. Seed candidates that fall outside the RoboCup field carpet are now rejected immediately, eliminating a class of impossible pose hypotheses that previously wasted particles.
- Introduced observation-scaled seed noise so that clusters spawned from distant observations are spread more widely (reflecting higher uncertainty), while clusters from close observations remain tight. This better represents the geometric uncertainty of far-away landmark triangulation.
- Created a `SeedCandidate` struct that bundles each candidate pose with its effective position noise. This lets downstream merging logic use observation-derived uncertainty rather than a fixed global spread parameter.
- Reserving a minimum of 10 percent of particle slots for uniformly-distributed particles in every seed event. This safety net guarantees that even if all seed candidates happen to be wrong, some particles remain spread across the field for future observation updates to find the correct mode.

- Added a mode-seeking fallback for the pose output when the particle set is multi-modal. When the bounding-box spread of the cloud exceeds 500 mm, the filter reports the highest-weighted single particle instead of the global weighted mean. This prevents the output from averaging two separated clusters into a nonsensical pose between them. For unimodal clouds (spread below threshold), the weighted mean is still used.
- Added configurable per-particle Gaussian noise to the motion update, with standard deviations exposed through the parameter file. This makes the noise model more flexible and tunable for different robot platforms.

Ghost Hypothesis Mitigation and Precision Loss Fixes

- Applied the particle's heading to the odometry displacement update, matching the same 2D rotation approach used by the existing MultiModalCMKF. Previously the odometry rotation was not being applied to the particle's local frame, causing systematic drift during turns.
- Reduced the roughening magnitude in `ParticleFilterParams.cfg` to prevent particles from jittering across the field's symmetric centre line when the filter had already converged. The roughening level is now just enough to prevent particle impoverishment without destroying localisation precision.
- Lowered `MinWeightAdjustment` from 0.01 to 0.001 so that low-weight particles in the incorrect symmetric mode decay faster between seed events. This helps the particle set collapse onto the correct hypothesis more quickly after a seed introduces competing modes.
- Suppressed stationary noise in the motion model: when the odometry displacement is zero (robot stationary), additive noise is also zero. Previously, particles continued to receive random jitter even when the robot was not moving, causing the cloud to slowly diverge from a correct localisation.
- Added a visibility hemisphere check in `updateUsingDirectionlessObservation()`. For observations that only provide distance information, the filter now checks whether the particle lies on a plausible side of the field relative to the feature's detectable hemisphere. This breaks X-axis symmetry for some directionless observations.

PR 339 -- Not Yet Merged

Title: "PAR: PF scale motion noises, directionless observation hemisphere check, multi mode killing" **Branch:** `par/feat/pf-mode-killing-v2` **Status:** Open

This PR builds on the changes already merged into `PAR/particle_filter_based_localisation` and introduces additional fixes for the symmetric localisation problem.

- **Fix A -- Zero-Stationary-Noise Motion Model.** Scales the additive noise in `motionUpdate()` by the odometry displacement magnitude. When odometry is zero, the additive noise is also zero, keeping the cloud tight when the robot is not moving so that particles cannot drift into the symmetric mode while stationary.
- **Fix B -- Reduced Roughening for a Converged Filter.** Lowers the roughening jitter in the config file. A converged stationary filter does not need 30 mm of positional jitter. The roughening is set only high enough to prevent particle impoverishment, not high enough to cross the field centre line.
- **Fix C -- Bimodal Cluster Detection and Mode Killing.** Adds a check after `observationUpdate()` but before `resampleParticles()`. If both sides of the field carry more than 25 percent of total

particle weight (bimodal distribution), the smaller cluster is killed by setting those particles to uniform weight and re-normalising. This directly mirrors the multi-modal pruning strategy used by the existing CMKF implementation.

- **Fix D -- Directionless Observation Hemisphere Check.** Tightens the `updateUsingDirectionlessObservation()` model by checking whether the robot's estimated position is on a plausible side of the field for each observed feature. Since directionless observations only provide distance, the X-axis symmetry can be broken by considering which features are visible from the robot's current hemisphere.
- **Fix E -- Faster Ghost Decay via Reduced `minWeightAdjustment`.** Further reduces `minWeightAdjustment` from 0.01 towards 0.001 or 0.0001 in the config. This makes wrong-mode particles decay faster between seed events, helping the filter converge on the correct hypothesis more quickly. This must be balanced against the increased risk of particle degeneracy.